

Contents

1	Extended Technical Appendix	1
1.1	Table of Contents	1
1.2	1. System Architecture Details	2
1.2.1	1.1 Component Inventory	2
1.2.2	1.2 Data Flow Diagram	2
1.2.3	1.3 Storage Requirements	3
1.3	2. Complete Algorithm Specifications	3
1.3.1	2.1 Hybrid Query Algorithm (Detailed)	3
1.3.2	2.2 Query Type Detection	7
1.4	3. MCP Directive Schema (JSON)	8
1.4.1	3.1 Complete JSON Schema	8
1.5	4. Configuration Reference	10
1.5.1	4.1 ChromaDB Configuration	10
1.5.2	4.2 Neo4j Configuration	10
1.5.3	4.3 MCP Server Configuration	10
1.6	5. Deployment Procedures	11
1.6.1	5.1 Fresh Deployment (Step-by-Step)	11
1.6.2	5.2 Update Deployment (CI/CD)	12
1.7	6. Performance Tuning Guide	12
1.7.1	6.1 Query Response Time Optimization	12
1.7.2	6.2 Memory Optimization	13
1.8	7. Troubleshooting Common Issues	13
1.8.1	7.1 ChromaDB Connection Errors	13
1.8.2	7.2 Neo4j Graph Query Timeouts	14
1.8.3	7.3 MCP Tool Returns Empty Results	14
1.9	8. Sample Query Results	14
1.9.1	8.1 Example Query: “How do I check for errors in production scripts?”	15
1.10	9. Reproducibility Checklist	16
1.10.1	9.1 Full System Reproduction	16
1.10.2	9.2 Minimal Reproduction (For Testing)	16

1 Extended Technical Appendix

Supplementary Material for “Spec-Driven Development with Supervised RAG Refinement”

Version: 1.0

Date: November 19, 2025

Purpose: Implementation details, configuration references, and reproducibility guidelines

1.1 Table of Contents

1. System Architecture Details
2. Complete Algorithm Specifications
3. MCP Directive Schema (JSON)
4. Configuration Reference
5. Deployment Procedures
6. Performance Tuning Guide
7. Troubleshooting Common Issues
8. Sample Query Results
9. Reproducibility Checklist

1.2 1. System Architecture Details

1.2.1 1.1 Component Inventory

Knowledge Base Components:

Component	Technology	Version	Purpose
Vector DB	ChromaDB	0.4.18	Semantic similarity search
Graph DB	Neo4j	5.13.0	Code structure relationships
Embedding Model	sentence-transformers/all-mpnet-base-v2	Latest	768-dim text embeddings
Code Parser	Tree-sitter	0.20.8	Multi-language AST parsing
MCP Server	UnifiedMCPServer.js	4.0.0	Tool orchestration

Infrastructure Services:

Service	Implementation	Port	Management
ChromaDB API	Docker container	8080	systemd (chromadb.service)
Neo4j Browser	Docker container	7474	systemd (neo4j.service)
Neo4j Bolt	Docker container	7687	systemd (neo4j.service)
MCP REST API	Node.js Express	3000	VS Code MCP integration

1.2.2 1.2 Data Flow Diagram

Code Repository (Git submodules: global-workflow, nws-hpc-standards)

```
> [Tree-sitter Parser]
    > AST extraction (functions, classes, imports)
    > Metadata extraction (LOC, language, module)

> [Neo4j Ingestion]
    > Graph relationships (CALLS, IMPORTS, DEFINES)

> [ChromaDB Ingestion]
    > Sentence Transformers (768-dim embeddings)
    > Graph metadata enrichment
    > Semantic annotation extraction (MCP directives)
```

User Query

- > [Query Type Detection] → Adjust weights (, ,)
- > [ChromaDB Search] → Top 50 semantic results
- > [Neo4j Traversal] → Graph context for entities
- > [Result Merging] → Hybrid scoring function
- > [Top-k Selection] → Return ranked results (k=10)

1.2.3 1.3 Storage Requirements

Disk Usage Breakdown:

```
# Knowledge base storage
/mcp_rag_eib/eib-mcp-rag-server/mcp_server_node/
  chromadb_data/      # 2.3 GB (5,307 documents × 768 dims)
  knowledge-base/     # 850 MB (cached documentation)
  logs/               # 120 MB (operational logs)

# Persistent database storage
/mcp_rag_eib/data/
  chromadb/           # 2.5 GB (persistent vector store)
  neo4j/              # 1.8 GB (graph database + indexes)

# Cache storage
/mcp_rag_eib/cache/
  transformers/       # 1.2 GB (MPNet model weights)
  npm/                # 450 MB (Node.js dependencies)
```

Total: ~9.2 GB

Memory Usage (Production):

Process	RSS	VSZ	Notes
ChromaDB	1.2 GB	3.8 GB	Increases with query volume
Neo4j	2.1 GB	4.5 GB	JVM heap + page cache
Node.js (MCP)	380 MB	1.2 GB	Stable under load
Total	~3.7 GB	~9.5 GB	8 GB RAM minimum, 16 GB recommended

1.3 2. Complete Algorithm Specifications

1.3.1 2.1 Hybrid Query Algorithm (Detailed)

```
def hybrid_query(query: str, options: dict) -> List[Result]:
    """
    Hybrid retrieval combining vector search, graph traversal, and semantic annotations.

    Args:
        query: Natural language query string
        options: {
            'max_results': int (default: 10),
            'content_type': str (default: 'all'),
```

```

        'threshold': float (default: 0.1),
        'include_graph': bool (default: True),
        'query_type': str (auto-detected)
    }

Returns:
    List of Result objects ranked by hybrid score
    """

# Step 1: Query type detection
query_type = detect_query_type(query)
weights = get_weights_for_query_type(query_type)
# weights = {'alpha': 0.5, 'beta': 0.3, 'gamma': 0.2} for default

# Step 2: Vector search (ChromaDB)
vector_results = chromadb.query(
    query_embeddings=[encode_text(query)],
    n_results=50,
    where={
        'content_type': options.get('content_type', 'all')
    },
    include=['metadatas', 'documents', 'distances']
)

# Step 3: Extract entities from query
entities = extract_entities(query)
# Example: "How do I use err_chk?" → [('err_chk', 'function')]

# Step 4: Graph traversal (Neo4j) if enabled
graph_results = []
if options.get('include_graph', True) and entities:
    for entity_name, entity_type in entities:
        if entity_type == 'function':
            # Find callers and callees
            cypher_query = """
            MATCH (f:Function {name: $name})
            OPTIONAL MATCH (f)-[:CALLS]-(caller:Function)
            OPTIONAL MATCH (f)-[:CALLS]->(callee:Function)
            OPTIONAL MATCH (file:File)-[:DEFINES]->(f)
            OPTIONAL MATCH (doc:Documentation)-[:DOCUMENTS]->(f)
            RETURN f,
                collect(DISTINCT caller.name) as callers,
                collect(DISTINCT callee.name) as callees,
                file.path as file_path,
                collect(DISTINCT doc.path) as docs
            """
            result = neo4j.run(cypher_query, name=entity_name)
            graph_results.extend(result)

        elif entity_type == 'file':
            # Find file dependencies
            cypher_query = """
            MATCH (f:File {path: $path})
            OPTIONAL MATCH (f)-[:IMPORTS]->(dep:File)

```

```

        OPTIONAL MATCH (importer:File)-[:IMPORTS]->(f)
        OPTIONAL MATCH (f)-[:DEFINES]->(func:Function)
        RETURN f,
            collect(DISTINCT dep.path) as dependencies,
            collect(DISTINCT importer.path) as importers,
            collect(DISTINCT func.name) as functions
    """
    result = neo4j.run(cypher_query, path=entity_name)
    graph_results.extend(result)

# Step 5: Extract annotation-matched results
    annotation_results = []
    for result in vector_results['metadatas'][0]:
        if 'annotation_type' in result:
            # Check if query intent matches annotation intent
            if matches_intent(query, result):
                annotation_results.append(result)

# Step 6: Merge results with hybrid scoring
    merged = merge_results(vector_results, graph_results, annotation_results)

# Step 7: Compute hybrid scores
    for result in merged:
        # Vector component (cosine similarity)
        vector_score = 1 - result.get('distance', 1.0) # ChromaDB returns distance

        # Graph component (structural relevance)
        graph_score = compute_graph_relevance(result, graph_results)

        # Annotation component (intent matching)
        annotation_score = compute_annotation_match(result, query)

        # Hybrid score
        result['hybrid_score'] = (
            weights['alpha'] * vector_score +
            weights['beta'] * graph_score +
            weights['gamma'] * annotation_score
        )

# Step 8: Sort by hybrid score and apply threshold
    ranked = sorted(
        merged,
        key=lambda x: x['hybrid_score'],
        reverse=True
    )
    filtered = [r for r in ranked if r['hybrid_score'] >= options.get('threshold', 0.1)]

# Step 9: Return top-k results
    return filtered[:options.get('max_results', 10)]

def compute_graph_relevance(result: dict, graph_results: list) -> float:
    """
    Compute structural relevance score based on graph position.

```

```

PageRank-inspired scoring:
- High if many other results link to this
- High if this is a hub (many outgoing links)
- High if this appears in multiple graph traversals
"""
score = 0.0

# Count appearances in graph results
appearances = sum(1 for gr in graph_results if result['id'] in gr.get('related', []))
score += min(appearances / 10.0, 1.0) * 0.4

# Check if it's a hub (many relationships)
relationship_count = len(result.get('calls_functions', [])) + len(result.get('imports_modules', []))
score += min(relationship_count / 20.0, 1.0) * 0.3

# Check if it's frequently called
caller_count = len(result.get('called_by_files', []))
score += min(caller_count / 10.0, 1.0) * 0.3

return score

def compute_annotation_match(result: dict, query: str) -> float:
    """
    Compute semantic annotation matching score.

    Checks if query intent aligns with document annotations.
    """
    score = 0.0

    # Check for annotation presence
    if 'annotation_type' not in result:
        return 0.0

    annot_type = result['annotation_type']

    # Intent matching
    if annot_type == 'mcp:intent':
        query_lower = query.lower()
        intent_keywords = {
            'how': 0.3,
            'use': 0.3,
            'why': 0.5,
            'purpose': 0.5,
            'reason': 0.5
        }
        for keyword, weight in intent_keywords.items():
            if keyword in query_lower:
                score += weight

    # Compliance matching
    elif annot_type == 'mcp:compliance':
        if 'compliant' in query_lower() or 'standard' in query_lower():

```

```

        score += 0.7
        # Boost if priority matches query urgency
        if 'critical' in query.lower() and result.get('priority') == 'critical':
            score += 0.3

    # Example matching
    elif annot_type == 'mcp:example':
        if 'example' in query.lower() or 'how to' in query.lower():
            score += 0.6

    # Pattern matching
    elif annot_type == 'mcp:pattern':
        if 'pattern' in query.lower() or 'best practice' in query.lower():
            score += 0.6

    return min(score, 1.0)

```

1.3.2 2.2 Query Type Detection

```

def detect_query_type(query: str) -> str:
    """
    Automatically detect query type to adjust retrieval weights.

    Returns: 'concept' / 'usage' / 'structure' / 'compliance' / 'troubleshoot'
    """
    query_lower = query.lower()

    # Concept queries
    concept_keywords = ['what is', 'what are', 'explain', 'describe', 'define']
    if any(kw in query_lower for kw in concept_keywords):
        return 'concept'

    # Code structure queries
    structure_keywords = ['calls', 'imports', 'depends on', 'uses', 'invokes', 'references']
    if any(kw in query_lower for kw in structure_keywords):
        return 'structure'

    # Compliance queries
    compliance_keywords = ['compliant', 'standard', 'ee2', 'required', 'mandatory']
    if any(kw in query_lower for kw in compliance_keywords):
        return 'compliance'

    # Troubleshooting queries
    troubleshoot_keywords = ['error', 'fail', 'issue', 'problem', 'why', 'broken']
    if any(kw in query_lower for kw in troubleshoot_keywords):
        return 'troubleshoot'

    # Usage queries (default)
    return 'usage'

def get_weights_for_query_type(query_type: str) -> dict:
    """
    Return optimized weights (alpha, beta, gamma) for query type.

```

```

"""
weights = {
    'concept': {'alpha': 0.7, 'beta': 0.1, 'gamma': 0.2},
    'usage': {'alpha': 0.5, 'beta': 0.3, 'gamma': 0.2},
    'structure': {'alpha': 0.2, 'beta': 0.7, 'gamma': 0.1},
    'compliance': {'alpha': 0.3, 'beta': 0.2, 'gamma': 0.5},
    'troubleshoot': {'alpha': 0.4, 'beta': 0.4, 'gamma': 0.2}
}
return weights.get(query_type, {'alpha': 0.5, 'beta': 0.3, 'gamma': 0.2})

```

1.4 3. MCP Directive Schema (JSON)

1.4.1 3.1 Complete JSON Schema

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "MCP Directive Schema",
  "description": "JSON schema for semantic annotations in RST documentation",
  "definitions": {
    "compliance": {
      "type": "object",
      "properties": {
        "directive": {"const": "mcp:compliance"},
        "category": {"type": "string"},
        "priority": {"enum": ["critical", "high", "medium", "low"]},
        "type": {"enum": ["mandatory", "recommended", "optional"]},
        "scope": {"enum": ["global", "system-specific", "component-specific"]}
      },
      "required": ["directive", "category", "priority", "type", "scope"]
    },
    "intent": {
      "type": "object",
      "properties": {
        "directive": {"const": "mcp:intent"},
        "identifier": {"type": "string"},
        "description": {"type": "string"},
        "enforcement": {"enum": ["runtime_check", "compile_check", "manual_review"]},
        "rationale": {"type": "string"}
      },
      "required": ["directive", "identifier", "description", "enforcement", "rationale"]
    },
    "severity": {
      "type": "object",
      "properties": {
        "directive": {"const": "mcp:severity"},
        "level": {"enum": ["must", "must-not", "should", "should-not", "may"]},
        "rationale": {"type": "string"},
        "exceptions": {"type": "string"}
      },
      "required": ["directive", "level", "rationale"]
    },
    "utility": {
      "type": "object",

```



```

    "properties": {
      "directive": {"const": "mcp:utility"},
      "name": {"type": "string"},
      "module": {"type": "string"},
      "category": {"type": "string"},
      "required": {"type": "boolean"},
      "deprecated": {"enum": ["yes", "no", "partial"]}
    },
    "required": ["directive", "name", "module", "category", "required", "deprecated"]
  },
  "example": {
    "type": "object",
    "properties": {
      "directive": {"const": "mcp:example"},
      "identifier": {"type": "string"},
      "language": {"type": "string"},
      "context": {"type": "string"},
      "demonstrates": {"type": "string"},
      "code": {"type": "string"}
    },
    "required": ["directive", "identifier", "language", "context", "demonstrates", "code"]
  },
  "pattern": {
    "type": "object",
    "properties": {
      "directive": {"const": "mcp:pattern"},
      "name": {"type": "string"},
      "category": {"type": "string"},
      "anti_pattern": {"type": "boolean"},
      "alternatives": {
        "type": "array",
        "items": {"type": "string"}
      },
      "description": {"type": "string"}
    },
    "required": ["directive", "name", "category", "anti_pattern"]
  },
  "see_also": {
    "type": "object",
    "properties": {
      "directive": {"const": "mcp:see-also"},
      "identifier": {"type": "string"},
      "related": {
        "type": "array",
        "items": {"type": "string"}
      },
      "type": {"enum": ["prerequisite", "reference", "alternative", "example"]}
    },
    "required": ["directive", "identifier", "related", "type"]
  }
},
"oneOf": [
  {"$ref": "#/definitions/compliance"},
  {"$ref": "#/definitions/intent"}
]

```

```

    {"$ref": "#/definitions/severity"},
    {"$ref": "#/definitions/utility"},
    {"$ref": "#/definitions/example"},
    {"$ref": "#/definitions/pattern"},
    {"$ref": "#/definitions/see_also"}
  ]
}

```

1.5 4. Configuration Reference

1.5.1 4.1 ChromaDB Configuration

File: /mcp_rag_eib/eib-mcp-rag-server/mcp_server_node/chromadb.yml

```

chroma:
  host: "localhost"
  port: 8080

  # Collection settings
  collection:
    name: "global-workflow-docs-v6-0-0-consolidated"
    embedding_function: "sentence-transformers/all-mpnet-base-v2"
    distance_metric: "cosine"

  # Performance tuning
  batch_size: 100
  max_batch_size: 5000

  # Persistence
  persist_directory: "/mcp_rag_eib/data/chromadb"

  # API settings
  allow_reset: false
  anonymized_telemetry: false

```

1.5.2 4.2 Neo4j Configuration

File: /mcp_rag_eib/eib-mcp-rag-server/SETUP/docker-compose.yml (Neo4j section)

```

neo4j:
  image: neo4j:5.13.0
  ports:
    - "7474:7474" # HTTP
    - "7687:7687" # Bolt
  environment:
    NEO4J_AUTH: neo4j/password
    NEO4J_dbms_memory_pagecache_size: 1G
    NEO4J_dbms_memory_heap_max_size: 2G
    NEO4J_dbms_security_procedures_unrestricted: apoc.*
  volumes:
    - /mcp_rag_eib/data/neo4j:/data

```

1.5.3 4.3 MCP Server Configuration

File: /mcp_rag_eib/eib-mcp-rag-server/mcp_server_node/mcp-config.env

```

# MCP Server Configuration
MCP_SERVER_VERSION=4.0.0
MCP_SERVER_MODE=full # Options: full, lite, github-only

# Database connections
CHROMADB_HOST=localhost
CHROMADB_PORT=8080
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=password

# Knowledge base paths
KNOWLEDGE_BASE_PATH=/mcp_rag_eib/eib-mcp-rag-server/mcp_server_node/knowledge-base
CHROMADB_DATA_PATH=/mcp_rag_eib/data/chromadb

# Performance tuning
MAX_QUERY_RESULTS=50
SEMANTIC_THRESHOLD=0.1
GRAPH_TRAVERSAL_DEPTH=3

# Health monitoring
HEALTH_CHECK_INTERVAL=60 # seconds
METRICS_RETENTION_DAYS=30

# Logging
LOG_LEVEL=info # Options: debug, info, warn, error
LOG_PATH=/mcp_rag_eib/eib-mcp-rag-server/mcp_server_node/logs

```

1.6 5. Deployment Procedures

1.6.1 5.1 Fresh Deployment (Step-by-Step)

```

# Step 1: Clone repository
cd /mcp_rag_eib
git clone https://github.com/TerrenceMcGuinness-NOAA/eib-mcp-rag-server.git
cd eib-mcp-rag-server

# Step 2: Initialize Git submodules
git submodule update --init --recursive

# Step 3: Run bootstrap script (idempotent)
cd SETUP
bash bootstrap.sh

# Step 4: Start services
systemctl --user start chromadb
systemctl --user start neo4j

# Step 5: Verify services
bash check-mcp-status.sh

# Step 6: Ingest documentation (first time only)
cd ../mcp_server_node

```

```
node scripts/ingest_documentation_week3.js
```

```
# Step 7: Build Neo4j graph
```

```
node scripts/build_code_graph.js
```

```
# Step 8: Start MCP server
```

```
bash start-mcp-server-node.sh
```

1.6.2 5.2 Update Deployment (CI/CD)

```
# Automated update script for production
```

```
# Step 1: Pull latest changes
```

```
cd /mcp_rag_eib/eib-mcp-rag-server
```

```
git fetch origin
```

```
git checkout MCP_node.js-RAG_ParallelWorks
```

```
git pull
```

```
# Step 2: Update submodules
```

```
git submodule update --remote
```

```
# Step 3: Install dependencies
```

```
cd mcp_server_node
```

```
npm install
```

```
# Step 4: Run incremental ingestion (only changed files)
```

```
node scripts/ingest_incremental.js
```

```
# Step 5: Update graph (incremental)
```

```
node scripts/update_code_graph_incremental.js
```

```
# Step 6: Restart MCP server
```

```
systemctl --user restart mcp-server
```

```
# Step 7: Validate deployment
```

```
bash test/health-check-mcp.sh
```

1.7 6. Performance Tuning Guide

1.7.1 6.1 Query Response Time Optimization

Target Metrics: - Concept queries: <500ms - Usage queries: <1000ms - Structure queries: <1500ms (graph traversal overhead) - Compliance queries: <800ms

Tuning Parameters:

Parameter	Default	Recommended Range	Impact
max_query_results	50	30-100	↑ = slower merge, ↓ = less recall
semantic_threshold	0.1	0.05-0.3	↑ = fewer results, faster
graph_traversal_depth	3	2-4	↑ = exponentially slower
chromadb.batch_size	100	50-500	↑ = faster ingestion, ↑ memory

Example Tuning for Production:

```
# High-traffic, low-latency priority
max_query_results: 30
semantic_threshold: 0.15
graph_traversal_depth: 2

# High-recall, research priority
max_query_results: 100
semantic_threshold: 0.05
graph_traversal_depth: 4
```

1.7.2 6.2 Memory Optimization

ChromaDB Memory Usage:

```
# Reduce memory footprint by limiting cached embeddings
chroma_client = chromadb.Client(Settings(
    chroma_db_impl="duckdb+parquet",
    persist_directory="/mcp_rag_eib/data/chromadb",
    anonymized_telemetry=False,
    allow_reset=False,
    chroma_server_grpc_port=None # Disable server mode if using Python API
))
```

Neo4j Memory Tuning:

```
# Edit Neo4j config for 16GB RAM system
NEO4J_dbms_memory_heap_max__size=4G
NEO4J_dbms_memory_pagecache_size=2G
```

1.8 7. Troubleshooting Common Issues

1.8.1 7.1 ChromaDB Connection Errors

Symptom: Connection refused to http://localhost:8080

Solutions:

```
# Check service status
systemctl --user status chromadb

# Check port binding
ss -tulpn | grep 8080

# View logs
journalctl --user -u chromadb -n 50

# Restart service
systemctl --user restart chromadb
```

Common Causes: - Port 8080 already in use → Change port in chromadb.yml - Docker container not running → docker ps to verify - Firewall blocking → sudo firewall-cmd --add-port=8080/tcp --permanent

1.8.2 7.2 Neo4j Graph Query Timeouts

Symptom: Query exceeded 30000 ms limit

Solutions:

```
-- Add indexes for frequent queries
CREATE INDEX function_name_index FOR (f:Function) ON (f.name);
CREATE INDEX file_path_index FOR (f:File) ON (f.path);

-- Verify indexes
SHOW INDEXES;

-- Check query plan
EXPLAIN MATCH (f:Function {name: 'err_chk'})<-[:CALLS]-(caller) RETURN caller;
```

Query Optimization:

```
-- BAD: Full graph traversal
MATCH (f:Function)-[:CALLS*..5]->(callee)
WHERE f.name = 'err_chk'
RETURN callee

-- GOOD: Limited depth with early termination
MATCH (f:Function {name: 'err_chk'})-[:CALLS*1..3]->(callee)
RETURN DISTINCT callee
LIMIT 50
```

1.8.3 7.3 MCP Tool Returns Empty Results

Symptom: search_documentation returns []

Diagnostic Steps:

```
// 1. Check collection exists
const collections = await chromaClient.listCollections();
console.log('Available collections:', collections.map(c => c.name));

// 2. Check document count
const collection = await chromaClient.getCollection({name: 'global-workflow-docs-v6-0-0-consolidated'});
const count = await collection.count();
console.log('Document count:', count);

// 3. Test basic query
const results = await collection.query({
  queryTexts: ['test'],
  nResults: 10
});
console.log('Query results:', results);
```

Common Fixes: - Collection name mismatch → Verify in `mcp-config.env` - Empty collection → Re-run ingestion script - Embedding model not loaded → Check transformers cache

1.9 8. Sample Query Results

1.9.1 8.1 Example Query: “How do I check for errors in production scripts?”

Query Type Detected: usage

Weights Applied: =0.5, =0.3, =0.2

Top 3 Results:

1.9.1.1 Result 1 (Score: 0.94)

```
{
  "document": "## Error Handling\n\n.. mcp:intent:: rapid_error_detection...",
  "metadata": {
    "file_path": "docs/EE2_standards.rst",
    "content_type": "documentation",
    "annotation_type": "mcp:intent",
    "priority": "critical",
    "related_utilities": ["err_chk", "err_exit"],
    "functions_mentioned": ["err_chk"]
  },
  "vector_score": 0.92,
  "graph_score": 0.85,
  "annotation_score": 0.90,
  "hybrid_score": 0.94
}
```

1.9.1.2 Result 2 (Score: 0.88)

```
{
  "document": ".. mcp:example:: err_chk_usage\n    :language: bash...",
  "metadata": {
    "file_path": "docs/production_utilities.rst",
    "content_type": "documentation",
    "annotation_type": "mcp:example",
    "demonstrates": "Standard error checking pattern",
    "context": "error_checking_after_command"
  },
  "vector_score": 0.89,
  "graph_score": 0.75,
  "annotation_score": 0.85,
  "hybrid_score": 0.88
}
```

1.9.1.3 Result 3 (Score: 0.81)

```
{
  "document": "def run_forecast():\n    ...\n    err_chk()\n    ...",
  "metadata": {
    "file_path": "scripts/exglobal_forecast.py",
    "content_type": "code",
    "functions_defined": ["run_forecast"],
    "calls_functions": ["err_chk", "prep_step"],
    "called_by_files": ["rocoto/forecast_job.xml"]
  },
  "vector_score": 0.78,
  "graph_score": 0.90, // High due to many callers
}
```

```
"annotation_score": 0.0, // Code has no annotations
"hybrid_score": 0.81
}
```

Response Time: 650ms (within target for usage queries)

1.10 9. Reproducibility Checklist

1.10.1 9.1 Full System Reproduction

To reproduce the complete SDD Framework deployment:

- ☐ **Hardware:** 8-core CPU, 16GB RAM, 100GB storage
- ☐ **OS:** Rocky Linux 8.6+ or Ubuntu 22.04+
- ☐ **Software:**
 - ☐ Git 2.30+
 - ☐ Docker 20.10+
 - ☐ Node.js 18.0+
 - ☐ Python 3.11+
 - ☐ systemd (for service management)
- ☐ **Network:** Access to GitHub, Docker Hub, Hugging Face Hub
- ☐ **Permissions:** User can run Docker, create systemd services

Step-by-Step:

1. Clone repository: `git clone <repo_url>`
2. Initialize submodules: `git submodule update --init --recursive`
3. Run bootstrap: `cd SETUP && bash bootstrap.sh`
4. Verify services: `bash check-mcp-status.sh`
5. Ingest data: `cd mcp_server_node && node scripts/ingest_documentation_week3.js`
6. Build graph: `node scripts/build_code_graph.js`
7. Start MCP: `bash start-mcp-server-node.sh`
8. Test: `bash test/health-check-mcp.sh`

Expected deployment time: 2-3 hours (including data ingestion)

1.10.2 9.2 Minimal Reproduction (For Testing)

To reproduce core functionality without full deployment:

```
# Install dependencies only
```

```
npm install
```

```
# Use in-memory databases (no Docker)
```

```
export CHROMADB_IMPL=in-memory
```

```
export NEO4J_IMPL=mock
```

```
# Run test suite
```

```
npm test
```

```
# Expected: All tests pass in <5 minutes
```

End of Extended Technical Appendix

Next Steps: - Compile LaTeX paper: `pdflatex SDD_Framework_Paper.tex` - Generate presentation: `pandoc SDD_Framework_Presentation.md -t beamer -o slides.pdf` - Train SMEs: Use `SME_Training_QuickStart.md`

For additional support: Terry.McGuinness@noaa.gov